

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – CODING CHALLENGE – ANSWERS

Course Name: Computer Networks **Course Code:** CSA07

Course Outcome: CO4 – Design network topologies (star, mesh, bus, hybrid) using networking devices, transmission media, and cabling standards

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
```

```

        self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
        break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)

```

```

print(f"Switch listening on {HOST}:{PORT}")
port_no = 1
while True:
    conn, addr = server.accept()
    threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
    port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return

```

```

        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()

```

```

topo.summary()
print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):

```

```

        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:

```

```

        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}

```



```

        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:

```

```

        length = self.cable_lengths[host]
        print(" " * 15 + "|")
        print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")
```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real

CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```
# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()
```

client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())
```

```
if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")
```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."
```

```

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):

```

```

        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```


client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")
```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
```

```

for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
    switch.add_port(host)

switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
switch.remove_port("Host-3")
switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are

registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
```

```

        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for
{backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")

```

```

        conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:

```

```

        print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host}>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

```

```
def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"
```

```

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

```



```

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

```

```

def display_ascii(self):
    print(f"\nStar Topology with {self.num_hosts} hosts")
    print(" " * 20 + f"[{self.switch_name}]")
    for host in self.hosts:
        length = self.cable_lengths[host]
        print(" " * 15 + "|")
        print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

def verify_connections(self):
    return all(host in self.cable_lengths for host in self.hosts)

def summary(self):
    print("\nHost\t\tSwitch\t\tCable Length (m)")
    for host in self.hosts:
        print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")
```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```
# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()
```

client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
```

```

        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame

is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
```

```

        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):

```



```

        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()

```

```

        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")

```

```

        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|" + f"{length}")
            print(f"{host}>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":

```

```
main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")
```

```

def transmit(self, sender, receiver):
    attempt = 1
    while True:
        self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")

```

```

        mac_table[src_mac] = port_no
        print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
        print("Updated MAC Address Table:")
        for mac, p in mac_table.items():
            print(f"  MAC: {mac} -> Port: {p}")
        conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

```

```

def remove_port(self, host_id):
    if host_id in self.ports:
        del self.ports[host_id]
        print(f"{host_id} disconnected from {self.name}")
    else:
        print(f"{host_id} not found on {self.name}")

def broadcast(self, source_host, frame):
    if source_host not in self.ports:
        print(f"Error: {source_host} is not connected to the switch")
        return
    recipients = [h for h in self.ports if h != source_host]
    self.broadcast_log.append((source_host, frame, recipients))
    print(f"\nBroadcast from {source_host}: '{frame}'")
    for host in recipients:
        print(f"  -> Delivered to {host}")
    return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

```

```

def summary(self):
    print("\nHost\t\tSwitch\t\tCable Length (m)")
    for host in self.hosts:
        print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation


```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by

inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
```

```

        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)

```

```
print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
```

```

network.simulate(5)
network.close()
print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"

```

```

PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")

```

```
switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
```

```

        attempt += 1
    else:
        self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
        break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```

server.bind((HOST, PORT))
server.listen(4)
print(f"Switch listening on {HOST}:{PORT}")
port_no = 1
while True:
    conn, addr = server.accept()
    threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
    port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:

```

```

        print(f"Error: {source_host} is not connected to the switch")
        return
    recipients = [h for h in self.ports if h != source_host]
    self.broadcast_log.append((source_host, frame, recipients))
    print(f"\nBroadcast from {source_host}: '{frame}'")
    for host in recipients:
        print(f"  -> Delivered to {host}")
    return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))

```

```

topo = StarTopology(n)
topo.display_ascii()
topo.summary()
print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

```

```

def log_event(self, message):
    timestamp = time.strftime("%H:%M:%S")
    line = f"[{timestamp}] {message}"
    print(line)
    self.log.write(line + "\n")

def transmit(self, sender, receiver):
    attempt = 1
    while True:
        self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

```

```

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):

```

```

        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")

```



```

        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")
```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real

CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```
# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()
```

client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())
```

```
if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")
```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."
```

```

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):

```

```

        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")
```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
```



```

for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
    switch.add_port(host)

switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
switch.remove_port("Host-3")
switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\t\tSwitch\t\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t\t{self.switch_name}\t\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are

registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
```

```

        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for
{backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")

```

```

        conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:

```

```

        print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host}>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

```

```
def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"
```

```

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

```

```

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class


```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

```

```

def display_ascii(self):
    print(f"\nStar Topology with {self.num_hosts} hosts")
    print(" " * 20 + f"[{self.switch_name}]")
    for host in self.hosts:
        length = self.cable_lengths[host]
        print(" " * 15 + "|" )
        print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

def verify_connections(self):
    return all(host in self.cable_lengths for host in self.hosts)

def summary(self):
    print("\nHost\t\tSwitch\t\tCable Length (m)")
    for host in self.hosts:
        print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")
```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```
# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()
```

client.py

```
# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
```

```

        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame

is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
```

```

        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):

```

```

        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()

```



```

        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")

```

```

        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|" + f"{length}")
            print(f"{host}>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":

```

```
main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")
```

```

def transmit(self, sender, receiver):
    attempt = 1
    while True:
        self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")

```

```

        mac_table[src_mac] = port_no
        print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
        print("Updated MAC Address Table:")
        for mac, p in mac_table.items():
            print(f"  MAC: {mac} -> Port: {p}")
        conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

```

```

def remove_port(self, host_id):
    if host_id in self.ports:
        del self.ports[host_id]
        print(f"{host_id} disconnected from {self.name}")
    else:
        print(f"{host_id} not found on {self.name}")

def broadcast(self, source_host, frame):
    if source_host not in self.ports:
        print(f"Error: {source_host} is not connected to the switch")
        return
    recipients = [h for h in self.ports if h != source_host]
    self.broadcast_log.append((source_host, frame, recipients))
    print(f"\nBroadcast from {source_host}: '{frame}'")
    for host in recipients:
        print(f"  -> Delivered to {host}")
    return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

```

```

def summary(self):
    print("\nHost\t\tSwitch\t\tCable Length (m)")
    for host in self.hosts:
        print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py


```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by

inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```
class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"  -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
```

```

        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)

```

```
print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
                attempt += 1
            else:
                self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
                break

    def simulate(self, num_transmissions=5):
        for _ in range(num_transmissions):
            sender, receiver = random.sample(self.hosts, 2)
            self.transmit(sender, receiver)

    def close(self):
        self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
```

```

network.simulate(5)
network.close()
print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"

```

```

PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")

```

```
switch.broadcast("Host-2", "DHCP Discover")
```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```
import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))
    topo = StarTopology(n)
    topo.display_ascii()
    topo.summary()
    print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()
```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```
def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")
```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```
import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

    def log_event(self, message):
        timestamp = time.strftime("%H:%M:%S")
        line = f"[{timestamp}] {message}"
        print(line)
        self.log.write(line + "\n")

    def transmit(self, sender, receiver):
        attempt = 1
        while True:
            self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
            if random.random() < 0.3: # 30% chance of collision
                backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
                self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
                time.sleep(0.05)
```



```

        attempt += 1
    else:
        self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
        break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to `csma_log.txt` as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```

server.bind((HOST, PORT))
server.listen(4)
print(f"Switch listening on {HOST}:{PORT}")
port_no = 1
while True:
    conn, addr = server.accept()
    threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
    port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:

```

```

        print(f"Error: {source_host} is not connected to the switch")
        return
    recipients = [h for h in self.ports if h != source_host]
    self.broadcast_log.append((source_host, frame, recipients))
    print(f"\nBroadcast from {source_host}: '{frame}'")
    for host in recipients:
        print(f"  -> Delivered to {host}")
    return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.

Question 1: Star Topology Generator

```

import random

class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-1"):
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        self.hosts = [f"Host-{i+1}" for i in range(num_hosts)]
        self.cable_lengths = {
            host: round(random.uniform(2.0, 50.0), 2) for host in self.hosts
        }

    def display_ascii(self):
        print(f"\nStar Topology with {self.num_hosts} hosts")
        print(" " * 20 + f"[{self.switch_name}]")
        for host in self.hosts:
            length = self.cable_lengths[host]
            print(" " * 15 + "|")
            print(f"{host:>15} ---- Cat6 UTP ({length} m) ---- {self.switch_name}")

    def verify_connections(self):
        return all(host in self.cable_lengths for host in self.hosts)

    def summary(self):
        print("\nHost\t\tSwitch\t\tCable Length (m)")
        for host in self.hosts:
            print(f"{host}\t\t{self.switch_name}\t\t{self.cable_lengths[host]}")

def main():
    n = int(input("Enter number of hosts: "))

```

```

topo = StarTopology(n)
topo.display_ascii()
topo.summary()
print("\nAll hosts connected:", topo.verify_connections())

if __name__ == "__main__":
    main()

```

Explanation

The program models a central switch and a configurable number of hosts, assigning each host a simulated Cat6 UTP cable length. Every host connects independently to the switch, so `verify_connections()` confirms all hosts are registered. A star topology gives reliable communication because each link is dedicated point-to-point: a cable or host fault affects only that one connection, the switch isolates collision domains per port, and centralised wiring makes fault isolation, monitoring, and expansion straightforward compared to a shared-medium bus topology.

Question 2: UTP Segment Length Validator

```

def validate_segment(length_m):
    MAX_LENGTH = 100 # IEEE 802.3 max UTP cable length in meters

    if length_m <= 0:
        return False, "Error: Cable length must be a positive value."
    if length_m > MAX_LENGTH:
        return False, f"Error: Cable length {length_m} m exceeds the IEEE 802.3 limit of {MAX_LENGTH} m."
    return True, f"Valid: Cable length {length_m} m is within the IEEE 802.3 limit."

if __name__ == "__main__":
    test_cases = [50, 100, 120]
    for length in test_cases:
        valid, message = validate_segment(length)
        print(f"Length: {length} m -> Valid: {valid} | {message}")

```

Test Case Results

50 m -> Valid (well within the 100 m IEEE 802.3 limit).

100 m -> Valid (exactly at the boundary, which the standard permits).

120 m -> Invalid (exceeds the 100 m limit; attenuation would degrade the signal beyond reliable operation).

Interpretation

The IEEE 802.3 standard caps UTP segment length at 100 m because signal attenuation and timing (round-trip collision-detection window) become unreliable beyond that distance. The function correctly separates valid from invalid lengths and returns a descriptive message for out-of-range values, which is useful for automated cabling audits.

Question 3: CSMA/CD Simulation

```

import random
import time

LOG_FILE = "csma_log.txt"

class CSMA_CD_Network:
    def __init__(self, hosts):
        self.hosts = hosts
        self.log = open(LOG_FILE, "w")

```

```

def log_event(self, message):
    timestamp = time.strftime("%H:%M:%S")
    line = f"[{timestamp}] {message}"
    print(line)
    self.log.write(line + "\n")

def transmit(self, sender, receiver):
    attempt = 1
    while True:
        self.log_event(f"{sender} attempting transmission to {receiver} (Attempt {attempt})")
        if random.random() < 0.3: # 30% chance of collision
            backoff = random.randint(1, 2 ** min(attempt, 10)) * 0.1
            self.log_event(f"Collision detected involving {sender}! Backing off for {backoff:.2f}s")
            time.sleep(0.05)
            attempt += 1
        else:
            self.log_event(f"{sender} -> {receiver}: Frame transmitted successfully")
            break

def simulate(self, num_transmissions=5):
    for _ in range(num_transmissions):
        sender, receiver = random.sample(self.hosts, 2)
        self.transmit(sender, receiver)

def close(self):
    self.log.close()

if __name__ == "__main__":
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    network = CSMA_CD_Network(hosts)
    network.simulate(5)
    network.close()
    print(f"\nSimulation complete. Log saved to {LOG_FILE}")

```

Explanation

Each simulated transmission first 'senses the carrier' by attempting to send; a random 30% chance of collision models two hosts transmitting simultaneously. On collision, the host calculates a random back-off interval that grows with the attempt number (an approximation of binary exponential back-off) and retries. Every attempt, collision, and back-off is timestamped and written to csma_log.txt as well as printed to the console. This mirrors real CSMA/CD behaviour, where collision detection and randomised back-off prevent hosts from repeatedly colliding in lockstep, reducing wasted bandwidth on a shared Ethernet segment.

Question 4: Client-Server Frame Exchange with MAC Learning

server.py

```

# server.py
import socket
import threading

HOST = "127.0.0.1"
PORT = 65000

mac_table = {} # {mac_address: port_number}

```

```

def handle_client(conn, addr, port_no):
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            src_mac, dest_mac, payload = frame.split("|")
            mac_table[src_mac] = port_no
            print(f"\nFrame received: SRC={src_mac} DEST={dest_mac} DATA={payload}")
            print("Updated MAC Address Table:")
            for mac, p in mac_table.items():
                print(f"  MAC: {mac} -> Port: {p}")
            conn.sendall(b"ACK")

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(4)
    print(f"Switch listening on {HOST}:{PORT}")
    port_no = 1
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr, port_no)).start()
        port_no += 1

if __name__ == "__main__":
    start_server()

```

client.py

```

# client.py
import socket

HOST = "127.0.0.1"
PORT = 65000

def send_frame(src_mac, dest_mac, payload):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac}|{dest_mac}|{payload}"
        s.sendall(frame.encode())
        response = s.recv(1024)
        print("Server response:", response.decode())

if __name__ == "__main__":
    send_frame("AA:BB:CC:00:00:01", "AA:BB:CC:00:00:02", "Hello from Host-1")

```

Explanation

The server plays the role of an Ethernet switch: each client connection is treated as a switch port. When a frame ('source MAC | destination MAC | payload') arrives, the server records the source MAC address against the port it arrived on, building a MAC address table dynamically — this is exactly how real switches learn addresses by inspecting the source field of incoming frames rather than being manually configured. On future frames, a switch can use this table to forward directly to the correct port instead of flooding all ports, reducing unnecessary traffic once every host's location is known.

Question 5: StarSwitch Class

```

class StarSwitch:
    def __init__(self, name="Switch-1"):

```

```

        self.name = name
        self.ports = {}          # {host_id: True}
        self.broadcast_log = []

    def add_port(self, host_id):
        self.ports[host_id] = True
        print(f"{host_id} connected to {self.name}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"{host_id} disconnected from {self.name}")
        else:
            print(f"{host_id} not found on {self.name}")

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"Error: {source_host} is not connected to the switch")
            return
        recipients = [h for h in self.ports if h != source_host]
        self.broadcast_log.append((source_host, frame, recipients))
        print(f"\nBroadcast from {source_host}: '{frame}'")
        for host in recipients:
            print(f"    -> Delivered to {host}")
        return recipients

if __name__ == "__main__":
    switch = StarSwitch()
    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)

    switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP Discover")

```

Explanation

The StarSwitch class encapsulates three core switch behaviours: `add_port()` and `remove_port()` manage which hosts are currently connected, and `broadcast()` delivers a frame to every connected port except the sender. Broadcasting is essential in Ethernet communication for scenarios where the destination is not yet known to the switch or the frame is inherently meant for all hosts — for example ARP requests ('who has this IP?') and DHCP discovery messages both rely on broadcast delivery so that the correct host can respond, after which the switch can learn the specific port for future unicast forwarding.